

Ingeniería de Software

La **ingeniería de software** es la disciplina que aplica principios de ingeniería al diseño, desarrollo, prueba, implementación y mantenimiento de programas informáticos. Su objetivo es crear software de alta calidad que sea confiable, eficiente, seguro y fácil de mantener.

En otras palabras, no solo se trata de programar, sino de seguir un proceso organizado para desarrollar aplicaciones y sistemas.

Las principales actividades de la ingeniería de software son:

- 1.**Análisis de requisitos:** Identificar las necesidades del usuario y definir qué debe hacer el software.
- 2.**Diseño:** Planificar la estructura, la arquitectura y el funcionamiento del sistema.
- 3.**Desarrollo (programación):** Escribir el código utilizando lenguajes de programación.
- 4.**Pruebas (Testing):** Verificar que el software funcione correctamente y corregir errores.
- 5.**Implementación:** Poner el software en funcionamiento para que los usuarios lo utilicen.
- 6.**Mantenimiento:** Actualizar, mejorar y corregir el software después de su lanzamiento.

Ejemplo:

Si una empresa quiere crear una aplicación para realizar compras en línea, un ingeniero de software:

- Analiza las necesidades de los clientes y la empresa.
- Diseña cómo funcionará la aplicación.
- Programa las funciones de compra y pago.
- Prueba que todo funcione correctamente.
- Publica la aplicación y la mantiene actualizada.

En resumen: la ingeniería de software es la rama de la ingeniería que se encarga de desarrollar y mantener programas informáticos de manera sistemática, eficiente y con estándares de calidad.

Rol del Ingeniero de Software

Un **ingeniero (especialista) de software** es un profesional que se encarga de **crear, diseñar, probar y mantener software** utilizando principios de ingeniería.

No se limita a escribir código. También busca que el software sea:

- **Funcional:** que haga lo que se necesita.
- **Seguro:** que proteja los datos y evite vulnerabilidades.
- **Eficiente:** que utilice adecuadamente los recursos.
- **Escalable:** que pueda crecer sin volverse difícil de manejar.
- **Mantenible:** que otros desarrolladores puedan entenderlo y modificarlo.

¿Qué hace un ingeniero de software?

Por ejemplo, si una empresa quiere crear una aplicación bancaria, el ingeniero de software puede:

1. Analizar qué necesitan los usuarios.
2. Diseñar la arquitectura de la aplicación.
3. Programar diferentes funcionalidades.
4. Trabajar con bases de datos y servidores.
5. Realizar pruebas y corregir errores.

6. Revisar la seguridad del sistema.
7. Mantener y mejorar la aplicación con el tiempo.

En pocas palabras: un programador se enfoca principalmente en escribir código, mientras que un **ingeniero de software** aborda el desarrollo del software de manera más amplia, considerando el diseño, la calidad, la seguridad, el mantenimiento y la evolución del sistema.

Ciclo de Vida del Software

El **ciclo de vida del software** es el conjunto de **etapas que atraviesa un software desde que surge la necesidad de crearlo hasta que deja de utilizarse**.

Las etapas principales son:

- 1.**Planificación:** Se identifica el problema, se establecen objetivos, recursos, costos y tiempos.
- 2.**Análisis de requisitos:** Se determina qué debe hacer el software y qué necesitan los usuarios.
- 3.**Diseño:** Se define cómo estará construido el sistema, su arquitectura, interfaz, base de datos, etc.
- 4.**Desarrollo:** Los programadores escriben el código y construyen el software.
- 5.**Pruebas:** Se verifica que el sistema funcione correctamente y se buscan errores.
- 6.**Implementación:** El software se instala y se pone a disposición de los usuarios.
- 7.**Mantenimiento:** Se corrigen errores, se agregan funcionalidades y se adapta el software a nuevos requisitos.
- 8.**Retiro:** Cuando el software deja de ser útil o es reemplazado, se discontinúa.

Ejemplo sencillo

Imaginemos que una empresa quiere crear una **aplicación para administrar una biblioteca**:

Planificación → Requisitos → Diseño → Desarrollo → Pruebas → Implementación → Mantenimiento → Retiro

El ciclo puede repetirse varias veces, especialmente cuando aparecen nuevos requisitos o problemas.

En resumen: el ciclo de vida del software permite **organizar y controlar el proceso de desarrollo**, desde la idea inicial hasta el final de su utilización.

Pensemos el **ciclo de vida del software** como un proceso que permite pasar de una idea o necesidad a un sistema funcionando y, eventualmente, a su reemplazo. Dependiendo del modelo utilizado — cascada, iterativo, incremental, ágil, etc.— las etapas pueden variar o repetirse, pero conceptualmente suelen ser estas:

1. Planificación

Es el momento en el que se define **qué problema queremos resolver y si vale la pena desarrollar el software.**

Se analizan cuestiones como:

- ¿Qué problema tiene la organización?
- ¿Quiénes van a utilizar el sistema?
- ¿Qué objetivos queremos alcanzar? (KPI) (Key Performance Indicator)
- ¿Cuánto tiempo y dinero tenemos?
- ¿Qué personas y tecnologías necesitamos?
- ¿Existen riesgos importantes?

Por ejemplo, una biblioteca quiere dejar de utilizar registros en papel y crear un sistema para administrar libros y préstamos.

En esta etapa se podría determinar que el objetivo es:

"Crear un sistema que permita registrar libros, usuarios, préstamos y devoluciones."

También se realiza una **estimación inicial de costos, tiempos y recursos.**

2. Análisis de requisitos

Una vez definido el problema, debemos determinar **exactamente qué debe hacer el software.**

Los requisitos pueden ser:

Requisitos funcionales: describen las funciones que debe realizar el sistema.

Por ejemplo:

- Registrar usuarios.
- Registrar libros.
- Realizar préstamos.
- Registrar devoluciones.
- Buscar libros.

Requisitos no funcionales: describen características o condiciones que debe cumplir el sistema.

Por ejemplo:

- El sistema debe responder en menos de 2 segundos.
- Los usuarios deben iniciar sesión.
- La información debe estar protegida.
- El sistema debe poder soportar 1.000 usuarios simultáneos.

Esta etapa es muy importante porque un error en los requisitos puede provocar que se desarrolle un software que **funciona correctamente, pero no resuelve el problema real**.

3. Diseño

En esta etapa se decide **cómo vamos a construir el software**.

Es parecido a hacer los planos de una casa antes de construirla.

Se toman decisiones sobre:

- Arquitectura del sistema.
- Base de datos.
- Interfaces.
- Componentes.
- Tecnologías.
- Comunicación entre sistemas.
- Seguridad.
- Organización del código.

4. Desarrollo o implementación (Programación)

Ahora comienza la **programación propiamente dicha**.

Los desarrolladores transforman los diseños y requisitos en código.

Por ejemplo, podrían utilizar:

- Java
- Python
- JavaScript/TypeScript
- C#
- C++
- SQL

En esta etapa también se utilizan herramientas como **Git** para controlar las diferentes versiones del código.

En proyectos grandes, el desarrollo suele dividirse en tareas más pequeñas. Por ejemplo:

Backend

└─ Registro de usuarios

- └ Registro de libros
- └ Gestión de préstamos

Frontend

- └ Pantalla de inicio
- └ Lista de libros
- └ Formulario de préstamos

El resultado de esta etapa es una versión del software que puede ejecutarse.

5. Pruebas (Testing)

Aquí se verifica que el software **funcione como debería**.

No se trata solamente de abrir la aplicación y comprobar si funciona. Existen diferentes tipos de pruebas.

Pruebas unitarias: verifican pequeñas partes del programa.

Por ejemplo:

¿La función que calcula la fecha de devolución funciona correctamente?

Pruebas de integración: verifican que diferentes componentes funcionen correctamente juntos.

Por ejemplo:

¿La aplicación puede comunicarse correctamente con la base de datos?

Pruebas de sistema: verifican el sistema completo.

Por ejemplo:

¿Un usuario puede iniciar sesión, buscar un libro y realizar un préstamo?

También pueden realizarse pruebas de:

- Seguridad.
- Rendimiento.
- Usabilidad.
- Compatibilidad.
-

Cuando se encuentra un error, normalmente se vuelve al desarrollo para **corregirlo y volver a probar**.

Por eso el ciclo no es estrictamente lineal.

6. Implementación o despliegue

Una vez que el software fue desarrollado y probado, llega el momento de **ponerlo en funcionamiento para los usuarios reales**.

Por ejemplo, una aplicación web podría desplegarse en un servidor o en la nube.

En esta etapa pueden realizarse tareas como:

- Configurar servidores.
- Crear bases de datos.
- Instalar el software.
- Configurar dominios.
- Configurar permisos.
- Migrar datos existentes.
- Capacitar a los usuarios.

Por ejemplo, si la biblioteca tenía 20.000 libros registrados en un sistema antiguo, habría que **migrar esos datos** al nuevo sistema.

7. Mantenimiento

El software no termina cuando se publica. De hecho, **gran parte de su vida comienza después del lanzamiento**.

Durante el mantenimiento pueden ocurrir diferentes cosas:

Corrección de errores

"Cuando intento devolver un libro, el sistema muestra una fecha incorrecta."

Se corrige el problema.

Adaptación

"La empresa cambió su sistema operativo o necesita conectarse con otro sistema."

El software debe adaptarse.

Mejoras

"Los usuarios quieren poder reservar libros."

Se agrega una nueva funcionalidad.

Actualizaciones

Se actualizan librerías, frameworks, sistemas operativos y componentes para mantener el software seguro y compatible.

Por eso el mantenimiento puede durar **muchos años**.

8. Retiro

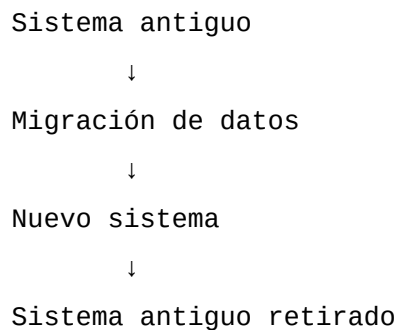
Finalmente, llega un momento en el que el software puede dejar de ser conveniente.

Puede ocurrir porque:

- La tecnología quedó obsoleta.
- Mantenerlo cuesta demasiado.
- Fue reemplazado por otro sistema.
- Ya no existen usuarios.
- La empresa cambió sus necesidades.

Antes de retirarlo, generalmente hay que planificar qué ocurre con los **datos**.

Por ejemplo:

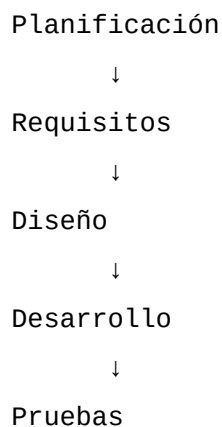


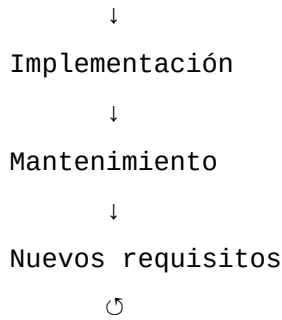
No necesariamente significa simplemente "apagar el servidor". Hay que asegurarse de que la información importante pueda conservarse o migrarse correctamente.

Lo importante: no siempre es lineal

Una idea fundamental es que estas etapas **no necesariamente ocurren una sola vez y en orden perfecto**.

Por ejemplo:





Durante el mantenimiento puede aparecer un nuevo requisito. Entonces volvemos a **análisis** → **diseño** → **desarrollo** → **pruebas** → **implementación**.

Esto es especialmente evidente en metodologías **ágiles**, donde el software se desarrolla mediante ciclos pequeños llamados **iteraciones o sprints**.

Una forma fácil de recordarlo

Podés pensar en el ciclo de vida como:

¿Qué necesitamos? → **¿Cómo lo vamos a hacer?** → **Lo construimos** → **¿Funciona?** → **Lo ponemos en producción** → **Lo mantenemos** → **Lo reemplazamos cuando sea necesario**.

Y una distinción importante para estudiar: **el ciclo de vida del software describe las etapas del proceso; el modelo de desarrollo (cascada, ágil, espiral, etc.) describe cómo se organizan y relacionan esas etapas**.